

Задание на практическую работу № 1

Тема работы: “Разработка кода через модульное тестирование”.

1. Необходимо спроектировать (небольшое, консольное) приложение для решения конкретной практической задачи, связанной с наукоемкой задачей (например, на графах).

Внимание! Недопустимо произвольное выполнение группового проекта, кроме оговоренных с преподавателем случаев, поэтому во избежание накладок по темам необходимо заранее согласовать тему информационной системы с преподавателем, согласованные названия проектов будут отражены в совместной таблице курса.

1.1. Проектирование случаев использования (use cases) должно включать:

- use-case диаграмму;
- меню и справочную информацию по приложению;
- ввод данных пользователем;
- загрузку данных пользователем (желательно использовать json, [пример входных данных 1](#), [пример входных данных 2](#)), должны приниматься файлы произвольного размера;
- решение задачи на графах/наукоемкой задачи и формирование вывода из программы в консоль и в файл (желательно использовать json, [пример выходных данных 1](#));
- формирование полезной рекомендательной информации для пользователя.

1.2. Решение **наукоемкой** задачи должно производиться не менее, чем двумя различными эффективными по времени и/или памяти способами (конкурирующими алгоритмами или одним алгоритмом с разными структурами хранения данных).

Примечание: допустимо выбирать в качестве проекта для тестирования разработанное автором ранее в рамках обучения в программе бакалавриата 09.03.03 программное обеспечение.

2. Изучите основные термины, подходы и методы тестирования (quality assurance (включая характеристики качества ПО), verification, validation, testing (с классификацией методов), mistake, fault, failure, error, test (item, case, suite, driver) [1,2,3,4].

Выберите две технологии проектирования тестов, основанной на спецификации [1,3]* из списка для проектирования тестов:

- boundary value analysis;
- equivalence partitioning;
- classification tree method;
- decision table testing.

Дополнительно выберите **две** технологии проектирования тестов, основанной на структуре ПО [1,3]* из списка для проектирования тестов:

- statement testing;
- branch testing;
- decision testing;
- branch condition testing;
- branch condition combination testing;
- modified condition/decision coverage (MCDC) testing;
- data flow testing.

Составьте для отчета:

- подробное описание выбранных технологий проектирования тестов;
- проверяемые условия (test cover items) для входной, выходной и внутренних процессов обработки данных;
- контрольные примеры (случаи тестирования, test cases).
- оптимизированные контрольные примеры, если это предполагает выбранный способ проектирования тестов.

Опишите преимущества и недостатки использованных технологий создания тестов. Сравните результаты покрытий проверяемых условий, количества контрольных примеров в сводной сравнительной таблице. Оцените трудозатратность методов.

3. Разработка (написание) приложения проводится с использованием модульных тестов по принципам TDD.

Вся функциональность классов должна покрываться модульными тестами (позитивные и негативные сценарии). В тестах должны быть реализованы контрольные примеры, спроектированные в предыдущей практической работе (четырьмя различными техниками проектирования тестов).

В процессе тестирования основного наукоемкого алгоритма должно применяться тестирование (сравнение) результатов алгоритма с другим (например другого типа перебора или использующий другую структуру данных), который был выбран для сравнения в первой практической задаче.

В процессе тестирования используйте функционал assertion (минимум 5 методов) и assumption (минимум 2 метода), мокирования (минимум 3 вида), параметризованных тестов, матчеров (минимум 2 вида) из JUnit5/Harmcrest или аналогичных библиотек на других языках.

В процессе написания ПО запрещено использовать многочисленные зависимости, редкие (в т.ч. платные) библиотеки, размер файла исходных файлов не должен превышать 20 Mb. Необходимые зависимости (например, для визуализации графов) должны быть согласованы с преподавателем.

4. Проверьте качество разработанных модульных тестов с помощью фреймворка мутационного тестирования (Pitest или аналога).

Опишите подробные значения мутации части исходного кода программы, который покрыт одним тестом.

Перечислите задействованные мутационные операторы.

Опишите, что такое эквивалентные мутации. Определите, были ли подобные мутации произведены при тестировании. Приведите пример.

Опишите значения метрик LCC, MSI, MCC, CoveredCodeMSI для разработанных модульных тестов. Определите, есть ли основания считать проверенные тесты mutation-adequate? Удалось ли улучшить качество тестов в результате мутационного тестирования, доработав тесты?

5. Оформите отчет и загрузите его в форму в курсе. Временем сдачи задания является время его защиты. Правила оценивания работы, оформления и сроки размещены в курсе.

Структура отчета:

- титульный лист (см. шаблон);
- содержание работы;
- введение;
- случаи использования приложения, включая use cases, функционал меню и др.;
- проектирование тестов (отдельно по каждому из 4х видов, выводы из сравнительного анализа),
- выбор и анализ использования библиотеки модульных тестов, основные используемые команды;
- описание реализации тестов по различным методикам проектирования тестов,
- описание реализации основного кода программы;
- описание мутационного тестирования;
- заключение, включающее описание:
 - краткие результаты проектирования приложения;
 - краткие результаты проектирования тестов;
 - преимущества и недостатки выбранной библиотеки модульного тестирования;
 - количества и качества разработанных тестов, подтвержденное различными метриками;
 - преимуществ и недостатков использования TDD в контексте Вашей работы;
- источники;
- приложение с текстом проекта (кода и тестов).

Рекомендуемая литература для выполнения задания (см. папку для самостоятельной работы учащихся, презентации).

1. "IEEE Standard Classification for Software Anomalies," in IEEE Std 1044-2009 (Revision of IEEE Std 1044-1993) , pp.1-23, 7 Jan. 2010, doi: 10.1109/IEEESTD.2010.5399061.
2. ISO/IEC 29119-1 Software and systems engineering -- Software testing - Part 1: Concepts and definitions.

3. ISO/IEC/IEEE 29119-2:2021 Software and systems engineering - Software testing - Part2: Test process.
4. ISO/IEC/IEEE 29119-4 Software and systems engineering - Software testing -- Part 4: Test techniques.
5. Лекции по дисциплине "Методы тестирования программного обеспечения", Пархоменко В.А., 2025, 2026.